

1 EGMN model (Q&A)

1.1 Q: What is EGMN in this repository?

A: EGMN (Exponential–Gaussian Mixture Network), implemented in `model/egmn.py`, predicts watch time by modeling a **mixture distribution** that captures short “quick-skip” behavior and longer watch-time behavior.

1.2 Q: What distribution family does EGMN use?

A: EGMN uses a mixture consisting of $1 \times$ Exponential (short peak near 0) plus $K \times$ Gaussian components for longer watches, implemented as Normals truncated below at 0, i.e. $p(y) = \pi_0 \text{Exp}(y; \lambda) + \sum_{k=1}^K \pi_k \text{TN}(y; \mu_k, \sigma_k, \text{lower} = 0)$ for $y \geq 0$.

1.3 Q: What does the architecture look like end-to-end?

A: The model encodes sparse/sequence/continuous features into a shared representation, feeds it through a shared MLP trunk, and uses separate heads to produce π logits, λ , μ , and σ , which parameterize the mixture used for training and prediction.

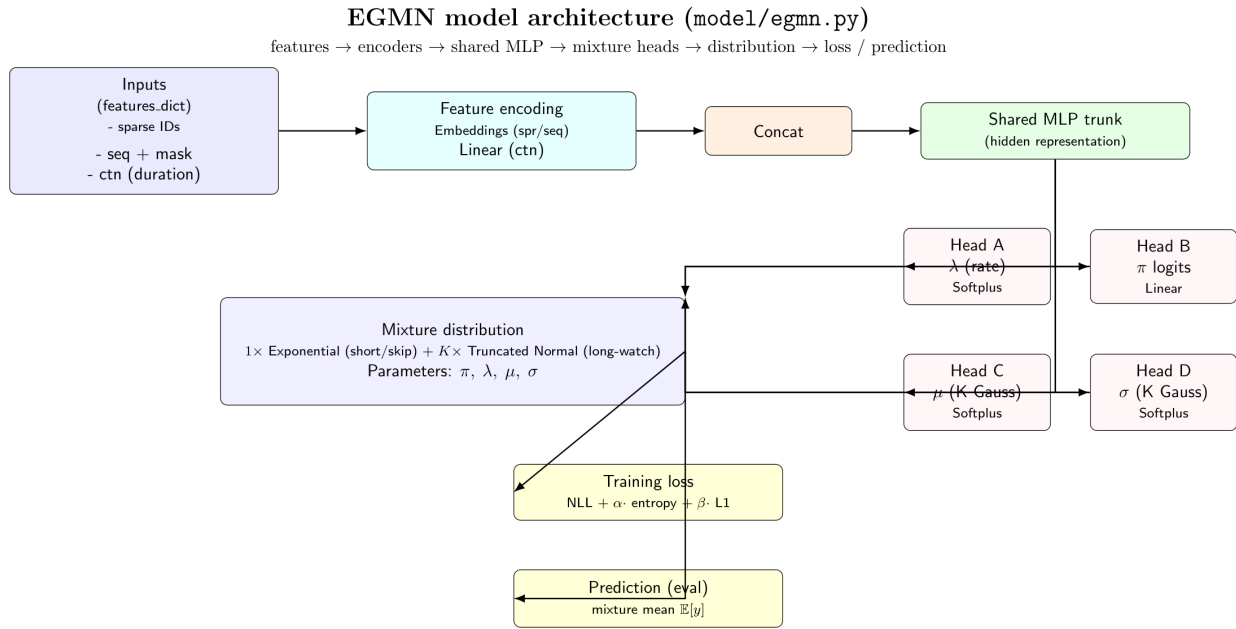


Figure 1: EGMN architecture overview: feature encoders + shared MLP + mixture-parameter heads, producing a distribution and a mean prediction.

1.4 Q: What parameters (heads) does EGMN output in code?

A:

```
# model/egmn.py (illustrative excerpt)
self.lambda_layer = nn.Sequential(
    nn.Linear(hidden_dim, 1),
    nn.Softplus(beta=0.5),
```

```
)

comp_num = 10 # number of Gaussian components
self.mixture_logits = nn.Linear(hidden_dim, comp_num + 1) # +1 for Exponential
self.gauss_mu = nn.Sequential(nn.Linear(hidden_dim, comp_num), nn.Softplus())
self.gauss_sigma = nn.Sequential(nn.Linear(hidden_dim, comp_num), nn.Softplus())
```

1.5 Q: What loss does EGMN train with in the run script?

A: Training minimizes a mixture negative log-likelihood plus regularizers: $\mathcal{L} = \mathcal{L}_{\text{NLL}} + \alpha \mathcal{L}_{\text{entropy}} + \beta \mathcal{L}_{\text{reg}}$, where $\mathcal{L}_{\text{reg}}$ is an L1 reconstruction between label and mixture-mean prediction and $\mathcal{L}_{\text{entropy}}$ is a mixture-weight entropy term.

1.6 Q: What scalar output is used for evaluation?

A: The evaluation uses a point estimate equal to the **mixture mean** (including the exponential mean $1/\lambda$), producing one scalar prediction per example.

1.7 Q: How is the mixture-mean prediction computed (schematically)?

A:

```
# model/egmn.py (illustrative excerpt)
pi, lambda_, mu, sigma = self.forward(features)
pi = torch.softmax(pi, dim=1)
component_means = torch.concat([1 / lambda_, mu], dim=1)
pred = torch.sum(pi * component_means, dim=1)
```